

AI LABS

Design System

Technical Documentation

Architecture, setup, component APIs, design tokens, accessibility, deployment, troubleshooting, and maintenance.

APPLICATION VERSION	0.1.0
FRAMEWORK	Angular 22.1
REVISION	21 September 2026
AUDIENCE	Users, developers, maintainers, and release engineers

Source of truth: checked-in Angular and CSS files in the application repository.

Contents

1. Purpose and scope
 2. System at a glance
 3. Repository structure
 4. Prerequisites and installation
 5. Using the showcase
 6. Consuming components in Angular
 7. Component API reference
 8. Design tokens and theming
 9. Application internals
 10. Accessibility guidance
 11. Building and deployment
 12. Extending the design system
 13. Quality assurance
 14. Security and privacy
 15. Troubleshooting
 16. Known limitations and roadmap priorities
 17. Maintenance checklist
- Appendix A. Public exports
- Appendix B. Configuration reference

Application version: 0.1.0 **Framework:** Angular 22.1 **Documentation revision:** 21 September 2026 **Repository:** [ItsVivekRamachandran/Ailabs-DesignSystem](#)

1. Purpose and scope

AI Labs Design System is a browser-based design-system showcase and component playground for AI-oriented products. It provides:

- CSS design tokens for color, spacing, radius, stroke, typography, shadow, and focus treatment.
- Standalone Angular components for common controls, navigation, search, date/time entry, and toolbars.
- Light and dark themes based on semantic token remapping.
- An interactive documentation site with component examples, token reference, live controls, and copyable Angular markup.

This document is intended for:

- **Application users**, who want to browse the design system and use the playground.
- **Angular developers**, who want to reuse or extend the components.
- **Design-system maintainers**, who need to understand the token and styling architecture.
- ~~Release engineers~~, who build and deploy the static site

Current packaging status: This repository is an Angular application and its package is marked `private`. It does not currently contain an Angular library build target or a publishable npm package. The site's "npm install @ailabs/design-system" copy action is demonstrative. Until library packaging is added, consume components from the repository source or deploy the showcase application.

2. System at a glance

2.1 Technology stack

Area	Technology	Repository version
Application framework	Angular	22.1.7
Build system / CLI	@angular/build , Angular CLI	22.1.8
Language	TypeScript	6.0.x
Reactive utilities	Angular signals, RxJS	RxJS 7.8.2
Runtime integration	Zone.js	0.16.x
Styling	Global CSS and CSS custom properties	Native browser CSS
Rendering	Client-side Angular application	Static output

There is no server-side API, database, authentication layer, analytics SDK, or remote data source in the current application.

2.2 High-level architecture

```
Browser
|
+-- index.html
|
+-- src/main.ts
|
+-- bootstraps standalone AppComponent
|
+-- documentation/showcase template
+-- Angular design-system components
+-- TypeScript token metadata
+-- global CSS token layer
+-- component CSS layer
+-- showcase layout CSS
+-- local SVG search assets
```

The application is deliberately small and uses no router, dependency-injection services, `NgModules`, or HTTP client. `AppComponent` is the composition root. Each design-system component is standalone and uses `ChangeDetectionStrategy.OnPush`.

2.3 Runtime data flow

1. `src/main.ts` imports `Zone.js` and bootstraps `AppComponent`.
2. Angular renders the single documentation page from `app.component.html`.
3. `AppComponent` imports token metadata from `tokens.ts` to render color, spacing, radius, and stroke references.
4. Components receive values through Angular inputs and emit user changes through Angular outputs.
5. The theme signal sets `data-theme` on the root HTML element. Semantic CSS variables then resolve to light or dark values.
6. The chosen theme is stored in browser `localStorage` under `ailabs-theme`.
7. An `IntersectionObserver` tracks the currently visible documentation section.
8. Clipboard actions use `navigator.clipboard` to copy install text, color values, or playground markup.

3. Repository structure

```
.
|-- angular.json Angular build and serve configuration
|-- index.html Host HTML document and metadata
|-- package.json Scripts and pinned application dependencies
|-- package-lock.json Reproducible npm dependency graph
|-- tsconfig.json Strict TypeScript/Angular compiler settings
|-- tsconfig.app.json Application compilation entry point
|-- docs/
|   |-- TECHNICAL_DOCUMENTATION.md Maintainer and user manual
|   |-- output/pdf/ Distributable PDF documentation
|   |-- src/
|       |-- main.ts Browser bootstrap
|       |-- styles.css Showcase/page layout and responsive rules
|       |-- assets/search/ Search component SVG icons
|       |-- design-system/
|           |-- tokens.css Runtime CSS custom properties and themes
|           |-- tokens.ts Typed token metadata used by the showcase
|           |-- components.css Shared styles for all ds-* components
|       |-- app/
|           |-- app.component.ts Page state and browser interactions
|           |-- app.component.html Documentation page and interactive playground
|       |-- design-system/
|           |-- index.ts Public TypeScript barrel exports
|           |-- *.component.ts Standalone Angular components
```

Generated production files are written to `dist/ailabs-design-system/browser/` and should not be edited manually.

4. Prerequisites and installation

4.1 Required software

Angular CLI 22.1.8 declares support for:

- Node.js `^22.22.3`, `^24.15.0`, or `>=26.0.0`.
- npm 8 or newer (the CLI also lists legacy compatible ranges).

The verified build documented here used Node.js 24.16.0 and npm 11.13.0.

4.2 Local setup

```
git clone https://github.com/ItsVivekRamachandran/Ailabs-DesignSystem.git
cd Ailabs-DesignSystem
npm ci
npm start
```

Open <http://localhost:4200/>. Angular's development server watches source files and reloads the page after changes.

Use `npm ci` in continuous integration and for clean reproducible installs. Use `npm install` when intentionally changing dependencies and updating `package-lock.json`.

4.3 Available commands

Command	Purpose
<code>npm start</code>	Start the development server (<code>ng serve</code>).
<code>npm run dev</code>	Alias for the development server.
<code>npm run build</code>	Create an optimized production build.
<code>npm run build:pages</code>	Build with <code>/Ailabs-DesignSystem/</code> as the base URL for GitHub Pages.
<code>npm run watch</code>	Continuously rebuild using the development configuration.

No repository scripts currently exist for unit tests, end-to-end tests, linting, formatting, Storybook, or npm package publication.

5. Using the showcase

5.1 Page navigation

The page has six tracked sections:

- Overview
- Foundations
- Colors
- Variables
- Components
- Playground

Selecting a navigation link updates the URL fragment without reloading the page and smoothly scrolls to the corresponding section. On initial load, a recognized URL fragment selects that section. While scrolling, `IntersectionObserver` updates the active navigation state.

5.2 Theme selection

Use the header theme control to switch between light and dark modes. The application:

- Updates the `data-theme` attribute on `<html>`.
- Updates the browser `theme-color` meta tag.
- Saves `light` or `dark` to `localStorage` as `ailabs-theme`.
- Defaults to light mode when the saved value is absent, invalid, or inaccessible.

The app does not currently derive the initial theme from `prefers-color-scheme`.

5.3 Component playground

The playground supports Button, Chip, Badge, Checkbox, Switch, Radio, Text field, Tabs, Menu item, List item, Search, and Toolbar. Choose a component in the left navigation and adjust its available controls. The preview and Angular markup update from signals and computed values.

The “Copy code” action writes the displayed markup to the clipboard and shows a temporary confirmation. Browser clipboard access may require HTTPS or localhost and user permission.

Date Calendar, Time Clock, Card, Tooltip, and Search Panel are demonstrated in the component catalog but are not selectable in the current playground menu.

6. Consuming components in Angular

6.1 Import pattern

All components are standalone and are re-exported from `src/app/design-system/index.ts`.

```
import { Component } from '@angular/core'
import {
  ButtonComponent,
  CheckboxComponent,
  InputComponent,
  ToolbarComponent,
} from './design-system'

@Component({
  selector: 'app-example',
  standalone: true,
  imports: [ButtonComponent, CheckboxComponent, InputComponent, ToolbarComponent],
  template: `
<ds-input
  label="Workspace name"
  placeholder="Enter a name"
  [(value)]="workspaceName"
/>

<ds-checkbox [(checked)]="rememberMe">Remember me</ds-checkbox>

<button dsButton (click)="save()">Save</button>
`,
})
export class ExampleComponent {
  workspaceName = ''
  rememberMe = false

  save(): void {
    // Application behavior belongs to the consumer.
  }
}
```

Angular's two-way binding syntax works where a component exposes an input and a corresponding `...Change` output, such as `value/valueChange` or `checked/checkedChange`.

6.2 Required global styles

Component templates rely on global CSS classes and variables. A consuming Angular application must load these files in this order:

```
{
  "styles": [
    "src/design-system/tokens.css",
    "src/design-system/components.css",
    "src/styles.css"
  ]
}
```

Only the first two files are required for the reusable component layer. `src/styles.css` belongs to the documentation showcase.

Because component styles are global, class names use the `ds-` prefix to reduce collisions. There is no Shadow DOM or Angular style encapsulation around the shared CSS.

6.3 Asset paths

Search components reference icons at `assets/search/*.svg`. A consuming app must copy `src/assets` into its output at `/assets`, or update the component templates to use the consumer's asset strategy.

7. Component API reference

7.1 Button

Import: `ButtonComponent` **Selector:** `button[dsButton]`

Input	Type	Default	Description
variant	'primary' 'secondary' 'outline' 'ghost'	'primary'	Visual hierarchy.
size	'sm' 'md' 'lg'	'md'	Control dimensions and text size.
iconOnly	boolean	false	Produces square icon-button sizing.

```
<button dsButton variant="secondary" size="lg">Generate</button>
<button dsButton [iconOnly]="true" aria-label="Add item">...</button>
```

Use the native `disabled`, `type`, click events, and ARIA attributes directly on the button. Icon-only buttons require an accessible name.

7.2 Badge

Import: `BadgeComponent` **Selector:** `ds-badge`

Input	Type	Default
tone	'neutral' 'violet' 'success' 'warning'	'neutral'
dot	boolean	false

```
<ds-badge tone="success" [dot]="true">Ready</ds-badge>
```

Badge content is projected. The optional dot is decorative and hidden from assistive technology.

7.3 Card

Import: CardComponent **Selector:** ds-card

Input	Type	Default
eyebrow	string	''
title	string	''
description	string	''

```
<ds-card eyebrow="Insights" title="Model health" description="Updated just now">
  <p>Projected card content</p>
</ds-card>
```

The header is omitted when all three header inputs are empty.

7.4 Checkbox

Import: CheckboxComponent **Selector:** ds-checkbox

Input / output	Type	Default / payload
checked	boolean	false
indeterminate	boolean	false
disabled	boolean	false
checkedChange	EventEmitter<boolean>	New checked value

```
<ds-checkbox [(checked)]="includeInsights">Include insights</ds-checkbox>
```

A user change clears `indeterminate`, updates the internal input value, and emits `checkedChange`. Each instance creates a unique input ID.

7.5 Chip

Import: ChipComponent **Selector:** button[dsChip]

Input	Type	Default
variant	'filled' 'outlined'	'filled'
selected	boolean	false
iconOnly	boolean	false

```
<button dsChip [selected]="active">AI ready</button>
```

The component provides styling only; consumers own selection state and click behavior. Supply `aria-pressed` when a chip acts as a toggle.

7.6 Text field

Imports: InputComponent or its export alias TextFieldComponent **Selectors:** ds-input, ds-text-field

Input / output	Type	Default / payload
label	string	''
hint	string	''
placeholder	string	''
type	string	'text'
icon	string	''; only 'search' renders an icon
value	string	''
error	string	''
disabled	boolean	false
valueChange	EventEmitter<string>	Current input text

```
<ds-input
label="Project name"
hint="Visible to collaborators"
placeholder="Enter project name"
[(value)]="projectName"
/>
```

When `error` is non-empty, it replaces the hint, sets `aria-invalid`, and links the input to the error text with `aria-describedby`.

7.7 Switch

Import: `SwitchComponent` **Selector:** `ds-switch`

Input / output	Type	Default / payload
checked	boolean	false
disabled	boolean	false
checkedChange	EventEmitter<boolean>	New checked value

The internal checkbox uses `role="switch"`. Prefer a switch for settings that take effect immediately.

7.8 Radio

Import: `RadioComponent` **Selector:** `ds-radio`

Input / output	Type	Default / payload
name	string	Generated group-like name
value	string	''
checked	boolean	false
disabled	boolean	false
checkedChange	EventEmitter<boolean>	Always <code>true</code> after user selection

```
<ds-radio name="model" value="fast" [checked]="model === 'fast'">Fast</ds-radio>
<ds-radio name="model" value="smart" [checked]="model === 'smart'">Smart</ds-radio>
```

Always provide the same explicit `name` for every option in a group. The component emits selection but not its `value`; the parent should associate the handler with the intended option.

7.9 Tab

Import: `TabComponent` **Selector:** `button[dsTab]`

Input	Type	Default
<code>active</code>	<code>boolean</code>	<code>false</code>

The host receives `role="tab"` and `aria-selected`. Place tabs in a `role="tablist"` container. The consumer must implement active-state changes, arrow-key navigation, `aria-controls`, and associated tab panels when building a complete tabs widget.

7.10 Tooltip

Import: `TooltipComponent` **Selector:** `ds-tooltip`

Input	Type	Default
<code>text</code>	<code>string</code>	<code>''</code>
<code>position</code>	<code>'top' 'bottom' 'left' 'right'</code>	<code>'top'</code>
<code>visible</code>	<code>boolean</code>	<code>false</code>

Projected trigger content receives `aria-describedby` through a wrapper. CSS displays the bubble on trigger hover or when `visible` is true. Consumers should also test keyboard focus and touch behavior for their usage context.

7.11 Menu item

Import: `MenuItemComponent` **Selector:** `button[dsMenuItem]`

Input	Type	Default
<code>shortcut</code>	<code>string</code>	<code>''</code>
<code>active</code>	<code>boolean</code>	<code>false</code>

The host receives `role="menuitem"`. Use inside an element with `role="menu"`. The consumer is responsible for menu opening/closing, roving focus, keyboard navigation, and action handling.

7.12 List item

Import: `ListItemComponent` **Selector:** `button[dsListItem]`

Input	Type	Default
<code>title</code>	<code>string</code>	<code>''</code>
<code>supportingText</code>	<code>string</code>	<code>''</code>
<code>selected</code>	<code>boolean</code>	<code>false</code>

The component has projection slots selected by `[avatar]`, `[title]`, and `[trailing]`.

```
<button dsListItem title="AI workspace" supportingText="Updated moments ago">
<span avatar>AW</span>
<span trailing aria-hidden="true"></span>
</button>
```

7.13 Search bar

Import: `SearchBarComponent` **Selector:** `ds-search-bar`

Input / output	Type	Default / payload
placeholder	string	'Hinted search text'
ariaLabel	string	'Search'
value	string	''
state	'default' 'hovered' 'focused'	'default'
valueChange	EventEmitter<string>	Current query
search	EventEmitter<string>	Submitted query
clear	EventEmitter<void>	Clear action

`state` is a showcase/visual-state control; actual browser hover and focus CSS also applies. Form submission prevents page navigation and emits the query.

7.14 Search panel

Import: `SearchPanelComponent` **Selector:** `ds-search-panel`

Input / output	Type	Default / payload
configuration	'input' 'supporting'	'input'
value	string	''
placeholder	string	'Hinted search text'
ariaLabel	string	'Search'
resultsLabel	string	'Search results'
results	readonly SearchResult[]	Built-in demo results when empty
valueChange	EventEmitter<string>	Current query
search	EventEmitter<string>	Submitted query
clear	EventEmitter<void>	Clear action
back	EventEmitter<void>	Back action
voice	EventEmitter<void>	Voice action
resultSelect	EventEmitter<SearchResult>	Selected result object

```
interface SearchResult {
  readonly id: string
  readonly label: string
  readonly supportingText: string
}
```

Pass real results explicitly in production. An empty array intentionally displays placeholder demo content rather than an empty state.

7.15 Date picker field

Import: `DatePickerComponent` Selector: `ds-date-picker`

Input / output	Type	Default / payload
label	string	'Label'
placeholder	string	'MM / DD / YYYY'
value	string	''
error	string	''
disabled	boolean	false
valueChange	EventEmitter<string>	Unparsed input string

This component is a styled text field. It does not parse, validate, localize, or open `DateCalendarComponent` automatically.

7.16 Date calendar

Import: `DateCalendarComponent` Selector: `ds-date-calendar`

Input / output	Type	Default / payload
view	'calendar' 'months' 'years'	'calendar'
selectedDay	number	24
selectedYear	number	2024
selectedDayChange	EventEmitter<number>	Selected current-month day

The present calendar is a fixed March demonstration: month labels, the 42-day grid, header text, and year list are static. Previous/next buttons have labels but no navigation handler. Month selection returns to the calendar view; year selection changes the year locally and returns to calendar view. Treat it as a visual prototype until full date logic is implemented.

7.17 Time picker field

Import: `TimePickerComponent` Selector: `ds-time-picker`

Its API mirrors the date field: `label`, `placeholder`, `value`, `error`, `disabled`, and `valueChange`. The default placeholder is `HH : MM`. Values are emitted as unparsed strings.

7.18 Time clock

Import: `TimeClockComponent` Selector: `ds-time-clock`

Input	Type	Default
mode	'dial' 'input'	'dial'
view	'hours' 'minutes'	'hours'
hour	string	'09'
minute	string	'30'

Input	Type	Default
period	'AM' 'PM'	'AM'
error	string	' '

The dial calculates twelve item positions at 30-degree intervals. Selecting an item changes the component's internal hour or minute. The component currently exposes no change outputs, and input mode displays labels rather than editable time inputs. It is therefore a visual/interaction prototype, not a complete form control.

7.19 Toolbar

Import: `ToolbarComponent` Selector: `ds-toolbar`

Input	Type	Default
variant	'docked' 'floating-horizontal' 'floating-vertical'	'docked'
tone	'standard' 'vibrant'	'standard'
expanded	boolean	true
ariaLabel	string	'Page actions'

Content projection uses `[toolbarLeading]`, the default slot, and `[toolbarTrailing]`. Leading and trailing groups always appear for the docked variant; floating variants hide them when collapsed. The internal container uses `role="toolbar"` and sets its orientation from the variant.

8. Design tokens and theming

8.1 Token layers

The token system has three conceptual layers:

- 1. **Primitive tokens** hold stable raw colors such as `--primitive-primary-500`.
- 2. **Semantic tokens** express intent such as `--text-primary`, `--surface-default`, or `--border-focus`.
- 3. **Stable component aliases** provide concise shared names such as `--color-ink`, `--space-4`, `--shadow-md`, and `--focus-ring`.

Components should prefer semantic or stable aliases over primitive colors. This allows a theme to change meaning without rewriting component CSS.

8.2 Color system

Primitive ramps include:

- Primary: 11 steps, 50 through 950.
- Secondary: 11 steps, 50 through 950.
- Support Gold: 11 steps, 50 through 950.
- Support Sand: 11 steps, 50 through 950.
- Neutral: 12 steps, 0 through 950.
- Black and White: 2 absolute values.

The TypeScript metadata describes 58 primitive color entries and 51 semantic color names across Background, Surface, Text, Primary action, Secondary action, Accent, Feedback, Icon, and Border groups.

8.3 Foundation scales

Scale	Values
Spacing	23 steps: 0, 1, 2, 3, 4, 6, 8, 10, 11, 12, 14, 16, 20, 24, 32, 40, 48, 56, 64, 80, 96, 120, 160 px
Radius	13 values from 0 px through 24 px, plus 100 px pill and 9999 px full
Stroke	0, 0.6667, 0.75, 1, 1.5, 2, 3, and 4 px
Font size	11 through 72 px across 13 named steps
Font weight	400, 500, 600, 700, and 800

8.4 Adding or changing a token

1. Add or change the CSS custom property in `src/design-system/tokens.css`.
2. For a theme-sensitive semantic token, define it in both `:root` and `:root[data-theme='dark']`.
3. If the documentation should enumerate it, update the matching metadata in `src/design-system/tokens.ts`.
4. Replace hardcoded component values with the semantic token where appropriate.
5. Inspect both themes and keyboard focus states.
6. Run the production build.

`tokens.css` and `tokens.ts` are maintained manually and are not generated from one another. Changes can drift unless both representations are reviewed.

9. Application internals

9.1 State model

`AppComponent` uses Angular signals for theme, current documentation section, copy feedback, selected playground component, and component controls. Computed values derive:

- The visible Light/Dark theme label.
- The selected playground component definition.
- Copyable Angular markup for the active playground state.

No application state leaves the browser. Theme preference is the only persisted value.

9.2 Browser APIs

API	Use	Fallback behavior
<code>localStorage</code>	Persist theme	Exceptions are caught; session still works.
<code>IntersectionObserver</code>	Active section tracking	No explicit fallback; navigation still scrolls when clicked.
History API	Replace URL fragment	Used without navigation/reload.
Clipboard API	Copy values and code	No error UI is currently implemented.
<code>scrollIntoView</code>	Smooth section navigation	Depends on browser support and motion settings.

9.3 Change detection

All components use `OnPush`. Inputs, outputs, user events, and Angular signals provide the relevant update boundaries. When adding mutable object inputs, prefer replacing the object or array reference so `OnPush` consumers update predictably.

9.4 Styling and responsiveness

The site uses global responsive CSS at 900 px and 600 px breakpoints; individual component adaptations occur at 520 px. The app honors `prefers-reduced-motion: reduce` in the showcase stylesheet. Modern CSS features include custom properties, `color-mix()`, `:has()`, `clamp()`, grid, and flexbox, so target browsers should be current evergreen releases.

10. Accessibility guidance

The implementation includes useful foundations: native form controls, linked labels, unique IDs, focus-visible styling, error relationships, `aria-invalid`, roles for tabs/menu/toolbar/search/listbox, and reduced-motion CSS.

Consumers must still complete widget-level behavior:

- Add accessible names to every icon-only button.
- Manage tab activation, focus movement, panels, and arrow-key behavior.
- Manage menu focus, escape handling, and open/close state.
- Set `aria-pressed` on selectable chips where appropriate.
- Announce asynchronous search status and empty states.
- Validate color contrast after changing tokens.
- Test keyboard-only, screen-reader, zoom, forced-color, reduced-motion, and touch use.
- Do not treat placeholders as labels.

The repository does not currently include automated accessibility tests or a published WCAG conformance statement.

11. Building and deployment

11.1 Production build

```
npm ci
npm run build
```

Output is written to:

```
dist/ailabs-design-system/browser/
```

The production configuration enables optimization, license extraction defaults, and filename hashing. Budgets are:

- Initial bundle warning at 500 kB and error at 1 MB.
- Any component style warning at 8 kB and error at 12 kB.

The build verified on 21 September 2026 completed successfully with a 307.30 kB raw initial bundle and a 75.10 kB estimated transfer size.

11.2 GitHub Pages build

```
npm run build:pages
```

This sets the base href to `/Ailabs-DesignSystem/`, matching the configured live demo path. Publish the contents of `dist/ailabs-design-system/browser/` through the repository's preferred GitHub Pages workflow.

For deployment at a different subpath, replace the base href:

```
npx ng build --base-href /your-subpath/
```

For a root-domain deployment, the regular `npm run build` output is appropriate.

11.3 Static hosting requirements

- Serve `index.html`, hashed JS/CSS bundles, and the `assets/` directory together.
- Use the correct base URL for subpath hosting.
- Serve over HTTPS if clipboard behavior is required outside localhost.
- Long-cache hashed assets; avoid permanently caching `index.html`.
- No server-side rewrite is currently required because the app has no Angular routes.

12. Extending the design system

12.1 Add a component

1. Create `src/app/design-system/<name>.component.ts`.
2. Use a `ds-` element selector or a `dsName` attribute selector.
3. Make the component standalone and use `ChangeDetectionStrategy.OnPush`.
4. Use semantic tokens and `ds-` prefixed classes in `components.css`.
5. Prefer native elements and semantics over recreated controls.
6. Add the export to `src/app/design-system/index.ts`.
7. Import the component in `AppComponent` if it will appear in the showcase.
8. Add a catalog example and, where appropriate, a playground definition and controls.
9. Document inputs, outputs, slots, keyboard interaction, and limitations.
10. Add tests before treating the component as production-ready.

12.2 API conventions

- Inputs use simple, explicit values and defaults.
- Two-way-bindable properties pair `property` with `propertyChange`.
- Native button attribute components preserve native events and attributes.
- Projected content provides labels and flexible icon/slot content.
- Host classes encode variants and state for global CSS.
- Component types are exported when consumers need them.

12.3 Turning the source into a publishable library

To distribute `@ailabs/design-system`, create an Angular library project rather than publishing this application package directly. The migration should:

1. Generate a library with Angular CLI and move reusable components into its source tree.
2. Define a public API that exports components and types.
3. Package token and component CSS as documented style entry points or component-scoped styles.
4. Move SVGs into an explicit asset contract or replace path-based assets with inline/component-owned icons.
5. Add peer dependencies for compatible Angular versions.

- 6. Add library builds, unit tests, linting, package metadata, versioning, and release automation.
- 7. Publish only after replacing the showcase's demonstrative install command with the real package name and tested installation instructions.

13. Quality assurance

13.1 Current verification

The repository currently relies on strict compilation and manual visual/interaction review. TypeScript enables strict mode, strict templates, strict injection parameters, `noImplicitReturns`, and related safety flags.

Run before every merge:

```
npm ci
npm run build
```

Then manually check:

- Light and dark themes.
- Desktop and mobile widths.
- Keyboard focus for every control.
- Every playground component and copy action.
- Search clear, submit, back, voice, and result events.
- Form error and disabled states.
- Date and time prototype interactions.
- GitHub Pages base-path asset loading.

13.2 Recommended test strategy

Layer	Recommended coverage
Unit	Defaults, host classes, input changes, output payloads, state methods.
Component integration	Content projection, two-way binding, ARIA relationships, event handling.
Accessibility	axe-based checks plus keyboard interaction tests.
Visual regression	Every component variant in both themes and responsive widths.
End-to-end	Navigation, theme persistence, playground controls, clipboard success/failure.
Build	Standard and GitHub Pages builds with bundle budgets.

14. Security and privacy

- The app does not collect, transmit, or store personal data on a server.
- The only persistent browser value is the theme preference.
- No secrets or environment variables are required at runtime.
- No user-supplied HTML is inserted into the DOM; Angular interpolation escapes text.
- Clipboard writes occur only after explicit UI actions.
- Dependencies should still be reviewed and updated through normal supply-chain controls.

If network features, analytics, or authentication are added later, update the threat model, privacy notice, Content Security Policy, and deployment documentation.

15. Troubleshooting

Build reports an unsupported Node.js version

Install a Node release accepted by Angular CLI 22.1.8, then reinstall dependencies with `npm ci`.

Search icons are missing

Confirm `src/assets` is included in the Angular `assets` configuration and that the application base href matches the deployment path.

Components render without styling

Load `tokens.css` before `components.css`. Components depend on global CSS variables and class rules.

Theme does not persist

Check whether the browser blocks local storage. The theme should still change for the current session even if persistence fails.

Copy buttons do not work

Run on localhost or HTTPS, allow clipboard access, and check browser developer tools. The current UI has no clipboard-error state.

GitHub Pages loads a blank or unstyled page

Use `npm run build:pages` and publish the `browser` directory contents. Confirm the repository/subpath casing exactly matches `/Ailabs-DesignSystem/`.

Calendar navigation does not change month

This is a known prototype limitation. The current calendar displays a fixed March grid and does not implement previous/next month logic.

Time selection does not update the parent form

`TimeClockComponent` currently changes only its internal inputs and exposes no outputs. Use `TimePickerComponent` for string entry or extend the clock with change events and a form-control contract.

16. Known limitations and roadmap priorities

The most important gaps before production library distribution are:

1. No publishable Angular library/package target despite the install command shown in the UI.
2. No automated unit, accessibility, visual, or end-to-end test suite.
3. Date Calendar is fixed to a March sample and lacks real navigation, parsing, localization, and complete keyboard support.
4. Time Clock lacks outputs and complete input-mode behavior.
5. Tabs and menu items provide semantics/styles but not composite-widget keyboard management.
6. Search Panel displays demo results when its `results` input is empty and has no explicit loading or empty states.
7. Clipboard failures are not surfaced to users.

8. CSS and TypeScript token definitions are manually duplicated.
9. No formal supported-browser matrix or automated browser compatibility testing exists.
10. No license file, contribution guide, changelog, or release policy is present in the repository.

Recommended priority order: package architecture, automated tests, date/time completion, accessibility hardening, token generation, then release governance.

17. Maintenance checklist

For every release or significant change:

- Update `package.json` version according to the release policy.
- Keep Angular packages on compatible versions.
- Run a clean install and both production build variants.
- Review output against bundle budgets.
- Verify component APIs and update this document.
- Check tokens in light and dark modes.
- Test keyboard and screen-reader behavior.
- Confirm the live demo base path and search assets.
- Record breaking changes and migration notes.
- Publish only artifacts produced from a clean, reviewed commit.

Appendix A. Public exports

The barrel file exports:

- `BadgeComponent`, `BadgeTone`
- `ButtonComponent`, `ButtonSize`, `ButtonVariant`
- `CardComponent`
- `CheckboxComponent`
- `ChipComponent`, `ChipVariant`
- `DateCalendarComponent`, `DatePickerComponent`, `CalendarView`
- `InputComponent` and alias `TextFieldComponent`
- `ListItemComponent`
- `MenuItemComponent`
- `RadioComponent`
- `SearchBarComponent`, `SearchPanelComponent`, `SearchConfiguration`, `SearchResult`, `SearchState`
- `SwitchComponent`
- `TabComponent`
- `TimeClockComponent`, `TimePickerComponent`, `ClockMode`, `ClockView`
- `ToolbarComponent`, `ToolbarTone`, `ToolbarVariant`
- `TooltipComponent`, `TooltipPosition`

Appendix B. Configuration reference

Angular build inputs

Setting	Value
Browser entry	<code>src/main.ts</code>
Host page	<code>index.html</code>
TypeScript config	<code>tsconfig.app.json</code>
Inline style language	CSS
Static asset source	<code>src/assets</code>
Default build	Production
Default serve mode	Development

Global stylesheet order

1. `src/design-system/tokens.css`
2. `src/design-system/components.css`
3. `src/styles.css`

This order is intentional: token definitions precede component rules, and showcase-specific rules load last.

Document ownership: Keep this document synchronized with component source, `package.json`, and `angular.json`. If code and documentation disagree, the checked-in source code is the operational authority.